# CasaTunes WebSocket Protocol Specification

**Version:** 1.12.0
**Date:** 2026-07-12
**Status:** Draft

---

## Table of Contents



---

## Overview

The CasaTunes WebSocket protocol enables clients to control and monitor a CasaTunes whole-house audio system. Communication is bidirectional: clients send **requests** and receive **responses**, and the server sends unsolicited **event broadcasts** to notify subscribed clients of state changes.

All messages are UTF-8 encoded JSON. The protocol is namespace-based, allowing logical grouping of operations (e.g., `avSwitch`, `mediaPlayer`) and fine-grained event subscriptions.

---

## Connection Lifecycle

Upon establishing a WebSocket connection, clients **must** follow this sequence:

1. `core.init` *(required)* — Identify the client to the server (name, version). Must be the first message sent after connecting.
2. `server.get` — Retrieve server configuration and status.

3. `avSwitch.zone.getAll` (system mode) or `avSwitch.stream.getAll` (streamer mode) — Retrieve all zones and inputs, or all streams. Check `server.get mode` to determine which call to make.
4. `mediaPlayer.getAll` — Retrieve now-playing state for all media players.
5. `core.subscribe` — Subscribe to event namespaces to keep state in sync.
6. **Issue operations as needed** — Rely on event broadcasts to update local state rather than polling.
7. **Respond to `core.ping`** — The server sends a ping every 60 seconds. The client must respond with `core.pong` within 10 seconds or the connection will be closed.

### State Reload

Clients must perform a full state reload — repeating steps 1–5 above — in the following situations:

- **On reconnect** — after a disconnection and successful re-establishment of the WebSocket connection, the client must reload all state as the server configuration or zone state may have changed while the client was offline.
- **On `server.changed` event** — when a `server.changed` event is received, the client must reload all state by repeating steps 2–5 (re-sending `core.init` is not required as the session is already established). This ensures the client reflects any changes to server configuration, mode, or available zones and inputs.

> **Note:** Clients should not assume that locally cached state remains valid after a reconnect or `server.changed` event. A full reload is the only safe way to guarantee consistency.

### Init Enforcement Rules

- `core.init` is **mandatory** and must be the first operation sent after connecting.
- The server allows a fixed window of **5 seconds** after the WebSocket connection is established for the client to send `core.init`. If the timeout expires without receiving `init`, the server closes the connection without sending a response.
- If a client sends **any other operation before `core.init`**, the server returns an error response and then immediately closes the connection.

**Error returned for premature operation:**

```
{
  "id":    "<echoed from request, or null if unparseable>",
  "ns":    "core",
  "op":    "init",
  "error": {
    "code":    403,
    "message": "core.init must be the first operation after connecting"
  }
}
```

---

## Message Envelope

## Request

Sent by the client to invoke an operation.

```
{
  "id":  "<string>",
  "ns":  "<string>",
  "op":  "<string>",
  "args": { }
}
```

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string | Yes | Client-defined identifier used to correlate responses. Any string format is acceptable. |
| `ns` | string | Yes | Namespace (e.g., `core`, `server`, `avSwitch`, `mediaPlayer`). |
| `op` | string | Yes | Operation name within the namespace (e.g., `zone.setVolume`). |
| `args` | object | No | Operation arguments. Omitted entirely when the operation takes no arguments. |

**Example — with args:**

```
{
  "id":   "42",
  "ns":   "avSwitch",
  "op":   "zone.setVolume",
  "args": { "zoneId": "12", "volume": 50 }
}
```

**Example — without args:**

```
{
  "id": "43",
  "ns": "server",
  "op": "get"
}
```

---

## Response

Sent by the server in reply to a client request.

```
{
  "id":   "<string>",
  "ns":   "<string>",
  "op":   "<string>",
  "data": { }
}
```

| Field | Type | Description |
|---|---|---|
| `id` | string | Echoes the `id` from the originating request. |
| `ns` | string | Echoes the `ns` from the originating request. |
| `op` | string | Echoes the `op` from the originating request. |
| `data` | object | Response payload. Fields that are unavailable or not applicable are omitted rather than set to `null`. |

**Example:**

```
{
  "id":   "42",
  "ns":   "avSwitch",
  "op":   "zone.setVolume",
  "data": { "zoneId": "12", "volume": 50 }
}
```

---

## Error Response

Returned instead of a normal response when the operation fails.

```
{
  "id":    "<string>",
  "ns":    "<string>",
  "op":    "<string>",
  "error": {
    "code":    <number>,
    "message": "<string>"
  }
}
```

| Field | Type | Description |
|---|---|---|
| `id` | string | Echoes the `id` from the originating request. |
| `ns` | string | Echoes the `ns` from the originating request. |

| Field | Type | Description |
|---|---|---|
| `op` | string | Echoes the `op` from the originating request. |
| `error.code` | number | Numeric error code. See Error Codes. |
| `error.message` | string | Human-readable description of the error. |

**Example:**

```
{
  "id":    "42",
  "ns":    "avSwitch",
  "op":    "zone.setVolume",
  "error": {
    "code":    404,
    "message": "ZoneID 12 does not exist"
  }
}
```

---

## Event Broadcast

Sent by the server to all subscribed clients when state changes. Not associated with any client request.

```
{
  "type":  "event",
  "ns":    "<string>",
  "event": "<string>",
  "data":  { }
}
```

| Field | Type | Description |
|---|---|---|
| `type` | string | Always `"event"`. Identifies this as an unsolicited broadcast. |
| `ns` | string | Namespace that originated the event. |
| `event` | string | Event name (e.g., `zone.changed`, `server.changed`). |
| `data` | object | Event payload. Always contains the complete current object. |

**Example:**

```
{
  "type":  "event",
  "ns":    "avSwitch",
  "event": "zone.changed",
```

```
  "data":  { "zoneId": "12", "volume": 40 }
}
```

> **Parsing tip:** Clients should check for `"type": "event"` first to distinguish event broadcasts from operation responses, since events do not carry an `id` or `op` field.

---

## Error Codes

| Code | Name | Description |
|---|---|---|
| 400 | Bad Request | The request message is malformed or missing required fields. |
| 403 | Forbidden | Operation rejected because `core.init` has not been completed. |
| 404 | Not Found | The specified resource (zone, input, etc.) does not exist. |
| 405 | Invalid Operation | The operation is not valid in the current server mode or state. |
| 409 | Conflict | The operation conflicts with the current state. |
| 422 | Invalid Argument | An argument value is out of range or of an unexpected type. |
| 500 | Internal Error | An unexpected server-side error occurred. |
| 503 | Service Unavailable | The server or a subsystem is not ready to handle requests. |

---

## Namespaces and Operations

### `core` Namespace

Core operations manage the client session and subscription state.

---

#### `core.init`

**Required.** Must be the first operation sent after connecting. Identifies the client to the server and establishes the session. The server allows a 5-second window after connection for `core.init` to arrive; if the timeout expires, the connection is closed. Any other operation sent before `core.init` results in a `403` error response followed by connection closure.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `clientName` | string | Yes | Human-readable name of the client application. |
| `clientVersion` | string | Yes | Version string of the client application. |

**Request:**

```
{
  "id":   "1",
  "ns":   "core",
  "op":   "init",
  "args": { "clientName": "Crestron Driver", "clientVersion": "2.1.0" }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `serviceVersion` | string | Version of the CasaTunes service. |

**Response:**

```
{
  "id":   "1",
  "ns":   "core",
  "op":   "init",
  "data": { "serviceVersion": "6.5.0" }
}
```

---

### `core.ping` / `core.pong`

The server sends a `core.ping` to each connected client every **60 seconds**. The client must respond with a `core.pong` within **10 seconds**. If no pong is received within the grace period, the server closes the connection.

**Server → Client ping:**

```
{
  "type":  "event",
  "ns":    "core",
  "event": "ping"
}
```

**Client → Server pong:**

```
{
  "id": "<any>",
  "ns": "core",
  "op": "pong"
}
```

**Server response to pong:** *(no response — pong is acknowledgement only)*

> **Note:** The ping is sent as an event broadcast (no `id`), since it is server-initiated. The client's pong uses the standard request envelope. Clients that fail to respond within 10 seconds will be disconnected.

---

### `core.subscribe`

Subscribes the client to one or more event topics. See Subscriptions for topic format.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `topics` | array of string | Yes | List of topic strings to subscribe to. |

**Request:**

```
{
  "id":   "3",
  "ns":   "core",
  "op":   "subscribe",
  "args": { "topics": ["avSwitch", "mediaPlayer"] }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `subscribed` | array of string | Topics that were successfully subscribed. |
| `warnings` | array of string | Topics that were not recognised and were ignored. Omitted if none. |

**Response:**

```
{
  "id":   "3",
  "ns":   "core",
  "op":   "subscribe",
  "data": {
    "subscribed": ["avSwitch", "mediaPlayer"]
  }
}
```

**Response (with unknown topic):**

```
{
  "id":   "3",
  "ns":   "core",
  "op":   "subscribe",
  "data": {
    "subscribed": ["avSwitch"],
    "warnings":   ["unknownNamespace"]
  }
}
```

---

### `core.unsubscribe`

Unsubscribes the client from one or more event topics.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `topics` | array of string | Yes | List of topic strings to unsubscribe from. |

**Request:**

```
{
  "id":   "4",
  "ns":   "core",
  "op":   "unsubscribe",
  "args": { "topics": ["mediaPlayer"] }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `unsubscribed` | array of string | Topics successfully unsubscribed. |
| `warnings` | array of string | Topics that were not recognised or not previously subscribed. Omitted if none. |

---

## `server` Namespace

Provides server identity, configuration, and status.

---

### `server.get`

Returns server information. Fields that are not applicable to the server's current mode are omitted from the response.

**Request:** *(no args)*

```
{ "id": "5", "ns": "server", "op": "get" }
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `name` | string | Server name. |
| `matrixModel` | string | Hardware model identifier. |
| `mac` | string | MAC address. |
| `ipAddress` | string | IP address. |
| `firmwareVersion` | string | Currently installed firmware version. |
| `latestFirmwareVersion` | string | Latest available firmware version. Omitted if no update information is available. |
| `mode` | string | Server mode: `"system"` or `"streamer"`. |
| `online` | boolean | `true` if the CasaTunes service is reachable and responding. |
| `ready` | boolean | `true` if the service is fully initialised and the WSS has a complete snapshot of all state (zones, inputs, now-playing). Clients should wait for `ready: true` before issuing control operations. |
| `startedOn` | string | ISO 8601 UTC timestamp of when the CasaTunes service started. |
| `sessionId` | string | Current session identifier. Changes when the server restarts or its configuration changes significantly. Clients should perform a full state reload when this value changes. |
| `numberOfStreams` | number | Number of streams available. Omitted in system mode. |
| `numberOfStreamsLicensed` | number | Number of licensed streams. |
| `numberOfZones` | number | Number of configured zones. Omitted in streamer mode. |
| `numberOfInputs` | number | Number of configured inputs. Omitted in streamer mode. |
| `problemList` | array | List of active problems. Each item is a ProblemItem object (see below). Empty array if none. |

**ProblemItem object:**

| Field | Type | Description |
|---|---|---|
| `problemId` | string | Unique identifier for this problem. |
| `code` | number | Numeric problem code. |
| `severity` | string | One of: `"info"`, `"warning"`, `"error"`. |
| `message` | string | Human-readable description of the problem. |

**Example:**

```
"problemList": [
  { "problemId": "FW001", "code": 100, "severity": "info",    "message":
"Firmware update available" },
  { "problemId": "NET001","code": 200, "severity": "warning", "message":
"Internet connection unavailable" },
  { "problemId": "ZN003", "code": 300, "severity": "error",   "message":
"Zone 3 is unreachable" }
]
```

---

### `server.message.respond`

Sends the user's response to a `server.message` event. Only valid while the message is still active.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `messageId` | string | Yes | The `messageId` from the `server.message` event. |
| `selection` | number | Yes | The `value` of the button selected by the user. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the response was accepted. |
| `error` | string | Human-readable error message. Omitted on success. |

---

### `server.tasks.get`

Returns the list of available tasks.

**Request:** *(no args)*

**Response data:**

| Field | Type | Description |
|---|---|---|
| `tasks` | array | Array of Task objects. |

**Task object:**

| Field | Type | Description |
|---|---|---|
| `taskId` | string | Unique task identifier. |
| `name` | string | Display name of the task. |

---

### `server.task.invoke`

Runs a task by name or ID.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `task` | string | Yes | Task name or task ID. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the task was successfully invoked. |

---

### `server.chimes.get`

Returns the list of available chime names. The returned names are used as the `chime` parameter in `server.chimes.play`.

**Request:** *(no args)*

**Response data:**

| Field | Type | Description |
|---|---|---|
| `chimes` | array | Array of chime name strings. |

**Example:**

```
{ "chimes": ["Chime 1", "Chime 2", "Westminster"] }
```

---

### `server.chimes.play`

Plays a chime. The REST endpoint called depends on which of `zoneId` and `chime` are provided:

| `zoneId` | `chime` | Behaviour |
|---|---|---|
| absent | absent | Plays the default chime in all rooms enabled for paging. |
| absent | present | Plays the specified chime in all rooms enabled for paging. |
| present | absent | Plays the default chime in the specified zone or zone group. |
| present | present | Plays the specified chime in the specified zone or zone group. |

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | No | Target zone or zone group — numeric ID or name. Omit to play in all paging-enabled rooms. |
| `chime` | string | No | Chime name from `server.chimes.get`. Omit to play the default chime. |
| `preWait` | number | No | Seconds to wait before playing (allows matrix hardware to power on). |
| `postWait` | number | No | Seconds to wait after playing (prevents the chime from being cut off). |
| `volume` | number | No | Override playback volume in the target zone or zone group. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the chime was successfully triggered. |

---

### `server.tts.play`

Plays text-to-speech audio. The REST endpoint called depends on whether `id` is provided:

| `id` | Behaviour |
|---|---|
| absent | `GET /system/tts/input/{input}` — plays in all rooms enabled for paging; voice params are optional. |
| present | `POST /system/tts` — plays in the specified zone or zone group; voice params are included in the request body. |

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `input` | string | Yes | The text to speak. |

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string | No | Target zone or zone group — numeric ID or name. Omit to play in all paging-enabled rooms. |
| `languageCode` | string | No | BCP-47 language code (e.g. `"en-US"`). |
| `gender` | string | No | Voice gender (e.g. `"MALE"`, `"FEMALE"`, `"NEUTRAL"`). |
| `voice` | string | No | Voice identifier (e.g. `"en-US-Wavenet-A"`). |
| `ssml` | boolean | No | `true` if `input` is SSML markup rather than plain text. |
| `preWait` | number | No | Seconds to wait before speaking. |
| `postWait` | number | No | Seconds to wait after speaking. |
| `volume` | number | No | Override playback volume. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the TTS request was accepted. |

---

### `server` Events

**`server.changed`** — Fired when server configuration or session changes. Clients must perform a full state reload when received. See State Reload.

```
{ "type": "event", "ns": "server", "event": "changed" }
```

---

**`server.issues.changed`** — Fired when the server problem list changes. `problemList` is `null` if all issues have been resolved.

```
{
  "type":  "event",
  "ns":    "server",
  "event": "issues.changed",
  "data":  {
    "problemList": [
      { "problemId": "NET001", "code": 200, "severity": "warning",
"message": "Internet connection unavailable" }
    ]
  }
}
```

---

`server.message` — Fired when the CasaTunes service sends a notification message to be displayed to the user. The client should display the message and, if buttons are present, allow the user to respond via `server.message.respond`. A `null` message means any currently displayed message should be dismissed.

```
{
  "type":  "event",
  "ns":    "server",
  "event": "message",
  "data":  {
    "messageId": "msg-001",
    "title":     "Firmware Update Available",
    "body":      "Version 6.6.0 is available. Would you like to update
now?",
    "duration":  30,
    "buttons": [
      { "value": 4, "title": "Yes" },
      { "value": 8, "title": "No"  }
    ]
  }
}
```

**Message data fields:**

| Field | Type | Description |
|---|---|---|
| `messageId` | string | Identifier used when responding via `server.message.respond`. |
| `title` | string | Message title. |
| `body` | string | Message body text. |
| `duration` | number | Suggested display duration in seconds. `0` means display indefinitely until dismissed. |
| `buttons` | array | Array of button objects. Omitted if no response is required. |

**Button object:**

| Field | Type | Description |
|---|---|---|
| `value` | number | Value to pass as `selection` in `server.message.respond`: `1` = OK, `2` = Cancel, `4` = Yes, `8` = No. |
| `title` | string | Display label: `"OK"`, `"Cancel"`, `"Yes"`, or `"No"`. |

---

## `avSwitch` Namespace

Controls AV switch zones and inputs (system mode) or streams (streamer mode). Check `server.get mode` to determine which set of operations to use — **do not mix zone and stream calls**.

> **ID identification in requests:** Any `zoneId` or `streamId` argument accepts either the numeric ID as a string (e.g. `"12"`) or the display name (e.g. `"Office"`, case-insensitive). IDs in responses and events are always the numeric ID string.

---

### Zone Operations (System Mode)

#### Data Shapes

**Input object:**

| Field | Type | Description |
|---|---|---|
| `inputId` | string | Unique identifier for the input. |
| `name` | string | Display name. |
| `type` | string | Input type. One of: `"casatunesMediaPlayer"`, `"sonosPlayer"`, `"unsupportedSource"`. |
| `hidden` | boolean | Whether the input is hidden from normal listings. |

**Zone object:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Numeric zone ID (e.g. `"12"`). Always numeric in responses and events. |
| `name` | string | Display name. |
| `hidden` | boolean | Whether the zone is hidden from normal listings. |
| `power` | boolean | `true` if powered on. |
| `mute` | boolean | `true` if muted. |
| `volume` | number | Current volume, 0–100. |
| `inputId` | string | Currently selected input. |
| `maxVolume` | number | Maximum allowed volume, 0–100. Always enforced. |
| `turnOnVolume` | object | `{ "value": number, "enabled": boolean }` — volume applied when the zone powers on. |
| `fixedVolume` | object | `{ "value": number, "enabled": boolean }` — fixed volume level. When enabled, volume controls are locked. |
| `zonesInGroup` | array | List of `zoneId` strings for each zone in the group. Empty if the zone is not grouped. |
| `sleepEnabled` | boolean | `true` if a sleep timer is currently active for this zone. |

**Example Zone object:**

```
{
  "zoneId":          "12",
  "name":            "Office",
  "hidden":          false,
  "power":           true,
  "mute":            false,
  "volume":          45,
  "inputId":         "2",
  "maxVolume":       80,
  "turnOnVolume":    { "value": 30, "enabled": true },
  "fixedVolume":     { "value": 50, "enabled": false },
  "zonesInGroup":    ["5", "9"],
  "sleepEnabled":    false
}
```

---

### `avSwitch.zone.getAll`

Returns all zones and inputs. Use in system mode only.

**Request:** *(no args)*

```
{ "id": "10", "ns": "avSwitch", "op": "zone.getAll" }
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zones` | array | Array of Zone objects. |
| `inputs` | array | Array of Input objects. |

---

### `avSwitch.zone.get`

Returns the full state of a single zone.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Zone to retrieve — numeric ID or name. |

**Response data:** A single Zone object.

---

### `avSwitch.zone.setPower`

Sets or toggles the power state of a zone.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone — numeric ID or name. |
| `power` | string | Yes | `"on"`, `"off"`, or `"toggle"`. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `power` | boolean | Resulting power state. |

---

### `avSwitch.zone.setMute`

Sets or toggles the mute state of a zone.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone — numeric ID or name. |
| `mute` | string | Yes | `"on"`, `"off"`, or `"toggle"`. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `mute` | boolean | Resulting mute state. |

---

### `avSwitch.zone.setVolume`

Sets the absolute volume of a zone.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone — numeric ID or name. |
| `volume` | number | Yes | Target volume, 0–100. Clamped to zone's `maxVolume` if exceeded. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |

| Field | Type | Description |
|---|---|---|
| `volume` | number | Resulting volume level. |

---

### `avSwitch.zone.adjustVolume`

Adjusts the volume of a zone by a signed delta. Positive values increase volume, negative values decrease it. The resulting volume is clamped to 0–100 and will not exceed the zone's `maxVolume`.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone — numeric ID or name. |
| `delta` | number | Yes | Amount to adjust. Positive to increase, negative to decrease (e.g., `5` or `-5`). |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `volume` | number | Resulting volume level after adjustment. |

**Example:**

```
{
  "id":   "15",
  "ns":   "avSwitch",
  "op":   "zone.adjustVolume",
  "args": { "zoneId": "12", "delta": -5 }
}
```

---

### `avSwitch.zone.setInput`

Sets the active input for a zone.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone. |
| `inputId` | string | Yes | Input to select. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `inputId` | string | Resulting active input. |

---

### `avSwitch.zone.setMaxVolume`

Sets the maximum volume ceiling for a zone. Always enforced — there is no enabled/disabled state.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone. |
| `value` | number | Yes | Maximum volume, 0–100. |

**Request:**

```
{
  "id":   "21",
  "ns":   "avSwitch",
  "op":   "zone.setMaxVolume",
  "args": { "zoneId": "12", "value": 80 }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `maxVolume` | number | Resulting max volume. |

---

### `avSwitch.zone.setTurnOnVolume`

Sets the volume applied when the zone powers on.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone. |
| `value` | number | Yes | Volume on power-on, 0–100. |
| `enabled` | boolean | Yes | `true` to apply the turn-on volume, `false` to disable it. |

**Request:**

```
{
  "id":   "22",
  "ns":   "avSwitch",
  "op":   "zone.setTurnOnVolume",
  "args": { "zoneId": "12", "value": 30, "enabled": true }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `turnOnVolume` | object | `{ "value": number, "enabled": boolean }` — resulting state. |

---

### `avSwitch.zone.setFixedVolume`

Sets a fixed volume level for a zone. When enabled, volume controls are locked at this level.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone. |
| `value` | number | Yes | Fixed volume level, 0–100. |
| `enabled` | boolean | Yes | `true` to lock volume at this level, `false` to disable. |

**Request:**

```
{
  "id":   "24",
  "ns":   "avSwitch",
  "op":   "zone.setFixedVolume",
  "args": { "zoneId": "12", "value": 50, "enabled": true }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `fixedVolume` | object | `{ "value": number, "enabled": boolean }` — resulting state. |

---

### `avSwitch.zone.group`

Links a zone to another zone, adding it to that zone's group. The zone is powered on and assigned the same source as the target zone. Returns `success: false` if CasaTunes cannot complete the join (e.g. the target zone is not powered on or does not exist).

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Zone to add to the group — numeric ID or name. |
| `targetZoneId` | string | Yes | Zone whose group to join — numeric ID or name. |

**Request:**

```
{
  "id":   "25",
  "ns":   "avSwitch",
  "op":   "zone.group",
  "args": { "zoneId": "9", "targetZoneId": "5" }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the zone was successfully linked. |

---

### `avSwitch.zone.ungroup`

Removes a zone from its group by powering it off.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Zone to remove from its group — numeric ID or name. |

**Request:**

```
{
  "id":   "26",
  "ns":   "avSwitch",
  "op":   "zone.ungroup",
  "args": { "zoneId": "9" }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the zone was successfully powered off. |

---

### `avSwitch.zone.setSleepTimer`

Powers off a zone with an optional delay. A `delay` of `0` powers the zone off immediately. A positive `delay` schedules the power-off after the specified number of seconds. The `sleepEnabled` field in the response reflects whether a sleep timer is now active.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `zoneId` | string | Yes | Target zone — numeric ID or name. |
| `delay` | number | Yes | Sleep delay in seconds. `0` = power off immediately. Must be ≥ 0. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `zoneId` | string | Target zone. |
| `power` | boolean | Resulting power state. |
| `sleepEnabled` | boolean | `true` if a sleep timer is now active. |

---

### Stream Operations (Streamer Mode)

In streamer mode each stream corresponds to a single audio output. Its `streamId` equals its `sourceId`. Streams are always powered on; volume may be fixed or variable. Use `avSwitch.stream.*` methods in streamer mode only.

**Stream object:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Numeric stream ID. Equals the corresponding `sourceId`. |
| `name` | string | Display name. |
| `hidden` | boolean | Whether the stream is hidden from normal listings. |
| `mute` | boolean | `true` if muted. |
| `volume` | number | Current volume, 0–100. |
| `maxVolume` | number | Maximum allowed volume, 0–100. |
| `fixedVolume` | object | `{ "value": number, "enabled": boolean }` — when enabled, volume controls are locked at this level. |

---

### `avSwitch.stream.getAll`

Returns all streams. Use in streamer mode only.

**Request:** *(no args)*

```
{ "id": "10", "ns": "avSwitch", "op": "stream.getAll" }
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streams` | array | Array of Stream objects. |

---

### `avSwitch.stream.get`

Returns the full state of a single stream.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Stream to retrieve — numeric ID or name. |

**Response data:** A single Stream object.

---

### `avSwitch.stream.setMute`

Sets or toggles the mute state of a stream.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Target stream — numeric ID or name. |
| `mute` | string | Yes | `"on"`, `"off"`, or `"toggle"`. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Target stream. |
| `mute` | boolean | Resulting mute state. |

---

### `avSwitch.stream.setVolume`

Sets the absolute volume of a stream.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Target stream — numeric ID or name. |
| `volume` | number | Yes | Target volume, 0–100. Clamped to stream's `maxVolume` if exceeded. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Target stream. |
| `volume` | number | Resulting volume level. |

---

### `avSwitch.stream.adjustVolume`

Adjusts the volume of a stream by a signed delta. The resulting volume is clamped to 0–100 and will not exceed the stream's `maxVolume`.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Target stream — numeric ID or name. |
| `delta` | number | Yes | Amount to adjust. Positive to increase, negative to decrease (e.g., `5` or `-5`). |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Target stream. |
| `volume` | number | Resulting volume level after adjustment. |

---

### `avSwitch.stream.setMaxVolume`

Sets the maximum volume ceiling for a stream.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Target stream — numeric ID or name. |
| `value` | number | Yes | Maximum volume, 0–100. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Target stream. |
| `maxVolume` | number | Resulting max volume. |

---

### `avSwitch.stream.setFixedVolume`

Sets a fixed volume level for a stream. When enabled, volume controls are locked at this level.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `streamId` | string | Yes | Target stream — numeric ID or name. |
| `value` | number | Yes | Fixed volume level, 0–100. |
| `enabled` | boolean | Yes | `true` to lock volume at this level, `false` to allow variable volume. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `streamId` | string | Target stream. |
| `fixedVolume` | object | `{ "value": number, "enabled": boolean }` — resulting state. |

---

### `avSwitch` Events

**`avSwitch.changed`** — Fired when any property of a zone or stream changes. `data` contains the complete Zone object (system mode) or Stream object (streamer mode).

```
{
  "type":  "event",
  "ns":    "avSwitch",
  "event": "changed",
  "data":  {
    "zoneId": "12", "name": "Office", "power": true, "volume": 40,
    "mute": false, "inputId": "2", "maxVolume": 80,
    "turnOnVolume": { "value": 30, "enabled": true },
    "fixedVolume": { "value": 0, "enabled": false },
    "zonesInGroup": ["5", "9"],
    "hidden": false, "sleepEnabled": false
  }
}
```

---

`avSwitch.group.changed` — Fired when group membership changes for one or more zones (zones are linked/unlinked or powered on/off). `data` contains an array of the affected groups in their updated state. A group whose last member has left will appear with an empty `zones` array.

```
{
  "type":  "event",
  "ns":    "avSwitch",
  "event": "group.changed",
  "data": {
    "groups": [
      { "sourceId": "2", "zoneIds": ["5", "9"] }
    ]
  }
}
```

**Group changed data fields:**

| Field | Type | Description |
|---|---|---|
| `groups` | array | Array of group objects for each group whose membership changed. |

**Group object:**

| Field | Type | Description |
|---|---|---|
| `sourceId` | string | The source shared by all zones in the group. |
| `zoneIds` | array | Array of `zoneId` strings. Empty if the group dissolved. |

---

`avSwitch.input.changed` — Fired when any property of an input changes. `data` contains the complete Input object.

```
{
  "type":  "event",
  "ns":    "avSwitch",
  "event": "input.changed",
  "data":  { "inputId": "2", "name": "Spotify", "type":
"casatunesMediaPlayer", "hidden": false }
}
```

---

## `mediaPlayer` Namespace

Controls media playback for audio sources (inputs).

**Data Shapes**

**NowPlaying object:**

| Field | Type | Description |
|---|---|---|
| `inputId` | string | Input this player is associated with. |
| `mediaId` | string | Identifier of the currently playing media item. |
| `artworkUrl` | string | URL of the current track or station artwork. Omitted if unavailable. |
| `metaData` | array | Ordered list of display strings describing the current track. Render in the order provided. See note below. |
| `serviceInfo` | object | Music service information. See below. |
| `searchPromptText` | string | Hint text for a search input scoped to the current source. Omitted if search is not available. |
| `duration` | number | Total track duration in seconds. `-1` if unavailable. |
| `progress` | number | Current playback position in seconds. `-1` if unavailable. |
| `status` | number | Playback status: `0` = stopped, `1` = paused, `2` = playing. |
| `featured` | object | `{ "isAvailable": boolean, "value": string }` — value is `"on"` or `"off"`. |
| `jump` | object | `{ "isAvailable": boolean }` — whether the user can jump forward/backward within the current track (e.g. podcast skip). |
| `next` | object | `{ "isAvailable": boolean, "isEnabled": boolean }` — `isAvailable` indicates the source supports skip-next; `isEnabled` indicates there is currently a next track to skip to. |
| `pause` | object | `{ "isAvailable": boolean }` — whether the pause action can be performed. |
| `play` | object | `{ "isAvailable": boolean }` — whether the play action can be performed. |
| `previous` | object | `{ "isAvailable": boolean, "isEnabled": boolean }` — `isAvailable` indicates the source supports skip-previous; `isEnabled` indicates skip-previous is currently actionable. |
| `progressBar` | object | `{ "isAvailable": boolean }` — whether a progress bar should be displayed for this source. |
| `queue` | object | `{ "isAvailable": boolean }` — whether the queue is supported for this source. |
| `repeat` | object | `{ "isAvailable": boolean, "value": string }` — value is `"on"`, `"once"`, or `"off"`. |
| `search` | object | `{ "isAvailable": boolean }` — whether search is available for this source. |

| Field | Type | Description |
|---|---|---|
| `seek` | object | `{ "isAvailable": boolean }` — whether the user can seek to an absolute position within the current track. |
| `shuffle` | object | `{ "isAvailable": boolean, "value": string }` — value is `"on"` or `"off"`. |
| `stop` | object | `{ "isAvailable": boolean }` — whether the stop action can be performed. |
| `thumbsDown` | object | `{ "isAvailable": boolean, "value": boolean }` — `isAvailable` indicates the source supports thumbs rating; `value` is `true` if the current track is rated down. |
| `thumbsUp` | object | `{ "isAvailable": boolean, "value": boolean }` — `isAvailable` indicates the source supports thumbs rating; `value` is `true` if the current track is rated up. |
| `hasQueueItems` | boolean | `true` if the queue contains one or more items. |

**Example NowPlaying object:**

```
{
  "inputId":    "1",
  "mediaId":    "12326726412843",
  "artworkUrl": "https://example.com/artwork.jpg",
  "metaData":   ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"],
  "serviceInfo": {
    "serviceName":    "Pandora",
    "serviceLogoUrl": "https://example.com/pandora-logo.png",
    "accountName":    "david@example.com",
  },
  "duration": 318,
  "progress": 42,
  "status":   2,
  "featured":    { "isAvailable": true,  "value": "on" },
  "jump":        { "isAvailable": false },
  "next":        { "isAvailable": true,  "isEnabled": true },
  "pause":       { "isAvailable": true },
  "play":        { "isAvailable": false },
  "previous":    { "isAvailable": true,  "isEnabled": false },
  "progressBar": { "isAvailable": true },
  "queue":       { "isAvailable": true },
  "repeat":      { "isAvailable": true,  "value": "on" },
  "search":      { "isAvailable": true },
  "seek":        { "isAvailable": true },
  "shuffle":     { "isAvailable": true,  "value": "off" },
  "stop":        { "isAvailable": true },
  "thumbsDown":  { "isAvailable": true,  "value": false },
  "thumbsUp":    { "isAvailable": true,  "value": false },
```

```
  "hasQueueItems": false
}
```

> **metaData note:** `metaData` is an ordered array of strings. Clients should display the items in the order provided without interpreting individual fields. The server will populate as many of the following as are available for the current service: song title, artist, album, playlist name. Items that are unavailable are omitted rather than included as empty strings.
>
> Example for a track with full metadata:
>
> ```
> "metaData": ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"]
> ```
>
> Example for a streaming radio station with only a station description:
>
> ```
> "metaData": ["Classic Rock Radio"]
> ```

**ServiceInfo object:**

| Field | Type | Description |
|---|---|---|
| `serviceName` | string | Name of the music service (e.g., `"Spotify"`). |
| `serviceLogoUrl` | string | URL of the service logo. |
| `accountName` | string | Name of the logged-in account. |

---

### `mediaPlayer.getAll`

Returns now-playing information for all media player inputs.

**Request:** *(no args)*

```
{ "id": "20", "ns": "mediaPlayer", "op": "getAll" }
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `players` | array | Array of NowPlaying objects, one per input. |

---

### `mediaPlayer.get`

Returns now-playing information for a specific input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Input to retrieve. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.play`

Starts playback on an input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.pause`

Pauses playback on an input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.playPause`

Toggles between play and pause. If currently playing, pauses; if paused or stopped, plays.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.stop`

Stops playback on an input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.next`

Skips to the next track.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.previous`

Returns to the previous track.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.shuffle`

Sets shuffle mode.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `mode` | string | Yes | `"on"` or `"off"`. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.repeat`

Sets the repeat mode.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `mode` | string | Yes | `"on"` (repeat all), `"once"` (repeat queue one more time), or `"off"`. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.featured`

Toggles or sets whether the current input is playing from featured content.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `mode` | string | Yes | `"on"`, `"off"`, or `"toggle"`. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.queue.clear`

Clears the playback queue.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.queue.get`

Returns the contents of the playback queue for an input. Supports pagination for large queues.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. Also used to compute each returned item's `canAdd` against the input's now-playing state. |
| `limit` | number | No | Maximum number of items to return. |
| `offset` | number | No | Zero-based start index for pagination. |

**Response data:** A single Queue object.

---

### `mediaPlayer.queue.save`

Saves the current playback queue as a named playlist.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `name` | string | Yes | Name for the new playlist. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the playlist was saved successfully. |
| `error` | string | Human-readable error message. Omitted on success. |

---

### `mediaPlayer.queue.playItem`

Starts playback from a specific position in the queue. The queue contents are unchanged — only the current play position advances. State updates are delivered via the `mediaPlayer.changed` event.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `index` | number | Yes | Zero-based index of the queue item to play. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the operation succeeded. |
| `error` | string | Human-readable error message. Omitted on success. |

---

### `mediaPlayer.queue.deleteItem`

Removes an item from the playback queue by index. The client may optimistically remove the item locally rather than re-fetching the full queue.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

| Field | Type | Required | Description |
|---|---|---|---|
| `index` | number | Yes | Zero-based index of the queue item to remove. |

**Response data:**

| Field | Type | Description |
|---|---|---|
| `success` | boolean | `true` if the item was removed successfully. |
| `error` | string | Human-readable error message. Omitted on success. |

---

### `mediaPlayer.thumbsUp`

Sends a thumbs-up rating for the currently playing track.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.thumbsDown`

Sends a thumbs-down rating for the currently playing track.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.position`

Seeks to an absolute position within the current track.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `position` | number | Yes | Absolute position in seconds from the start of the track. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.jump`

Adjusts the playback position by a relative offset. Positive values jump forward, negative values jump backward.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `delta` | number | Yes | Relative offset in seconds. Positive to jump forward, negative to jump backward. |

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.featured.get`

Returns featured or curated media items for an input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `start` | number | No | Zero-based offset for pagination. Defaults to `0`. |
| `maxItems` | number | No | Maximum number of items to return. |

**Request:**

```
{
  "id":   "30",
  "ns":   "mediaPlayer",
  "op":   "featured.get",
  "args": { "inputId": "1", "start": 0, "maxItems": 20 }
}
```

**Response data:**

| Field | Type | Description |
|---|---|---|
| `featuredItems` | array | Array of FeaturedItem objects. |

**FeaturedItem object:**

| Field | Type | Description |
|---|---|---|
| `mediaId` | string | Media identifier, usable with `mediaPlayer.media.play`. |

| Field | Type | Description |
|---|---|---|
| `title` | string | Display title (e.g., `"Today's Hits"`). |
| `type` | string | Content type description (e.g., `"Pandora Station"`, `"Spotify Playlist"`). |
| `artworkUrl` | string | URL of the artwork to display for this item. Omitted if unavailable. |

**Collection actions (`collectionActions`)**

`mediaPlayer.media.getRoot`, `mediaPlayer.media.getCollection`, and `mediaPlayer.media.search` all accept an optional `collectionActions` number argument — a bitmask that inserts ready-made list entries into the returned collection for whole-collection playback and per-item queueing, so the client doesn't need to build separate affordances for these actions itself.

**Bit values (combine with bitwise OR / addition):**

| Bit | Value | Effect |
|---|---|---|
| Play All | `1` | If the collection's `canPlay` is `true`, inserts a "Play All" item. |
| Shuffle All | `2` | If `canPlay` is `true`, inserts a "Shuffle All" item. |
| Add To Queue | `4` | If `canAddToQueue` is `true`, inserts an "Add All To Queue" item. If **both** this bit and Play All (`1`) are set, any item in the collection that supports queueing (`canAdd: true`) additionally becomes browsable — navigating into it returns a small two-item menu, "Play Now" and "Add To Queue" — letting a client that only supports single-tap selection still choose how to handle that item. |

Omit the argument or pass `0` for none. Each bit is independent — for example, `2` alone requests just "Shuffle All" with no "Play All"; `4` alone requests only "Add All To Queue" with no per-item menu (that needs bit `1` too); `5` (`1 + 4`) requests "Play All" and "Add All To Queue" *and* the per-item menu, without "Shuffle All". Any value with bits outside `1|2|4` (i.e. outside `0`–`7`) returns an `InvalidParams` error.

These items behave like any other MediaItem returned by this API — treat their `mediaId` as opaque and select them the normal way, via `mediaPlayer.media.play`. `addToQueue` does not need to be supplied when selecting one of them; the server already knows the correct action.

For `getCollection`/`search`, which paginate via `offset`, the "Play All"/"Shuffle All"/"Add All To Queue" items are only inserted on the first page (`offset` null or `0`) to avoid repeating them on every scrolled-in page. The per-item "Play Now"/"Add To Queue" menu applies on every page.

---

### `mediaPlayer.media.getRoot`

Returns the top-level browsable collection for an input. This is the entry point for all media browsing — call this first to get the root menu of services, playlists, bookmarks, and history available for the input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `includePlaylists` | boolean | No | Whether to include playlists. |
| `maxPlaylists` | number | No | Maximum number of playlists to return. |
| `includeOtherPlaylists` | boolean | No | Whether to include playlists from other sources. |
| `maxBookmarks` | number | No | Maximum number of bookmarks to return. |
| `includeSelectionHistory` | boolean | No | Whether to include recently played items. |
| `collectionActions` | number | No | See Collection actions. Default `0`. |

**Response data:** A single Collection object.

---

### `mediaPlayer.media.getCollection`

Browses into a collection item. Call this with a `mediaId` returned from `media.getRoot` or a previous `media.getCollection` to navigate deeper into the media hierarchy. Supports pagination via `limit` and `offset`.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Collection to browse into (from a previous browse response). |
| `inputId` | string | No | Input currently in use. When provided, the server uses the input's now-playing state to compute `canAddToQueue`. |
| `limit` | number | No | Maximum number of items to return. |
| `offset` | number | No | Zero-based start index for pagination. |
| `enableFormProcessing` | boolean | No | Default `false`. When `false`, if the server returns a form (e.g. for authentication), the form is suppressed and a single informational MediaItem is appended to the items list instead. Its title is `"{collection.title} using CasaTunesX"`, or `"Check the CasaTunesX App for more information"` if the collection has no title. All capability flags on this item are `false` — it cannot be played, selected, or added to the queue. Set to `true` only for UIs that implement full form handling. |

| Field | Type | Required | Description |
|---|---|---|---|
| `collectionActions` | number | No | See Collection actions. Default `0`. Applied only when `offset` is null or `0`. |

**Response data:** A single Collection object. If the underlying collection could not be found or has no items (and isn't a form or error response), a placeholder Collection is returned instead: all capability flags `false`, and a single non-playable MediaItem titled `"No items found"`.

---

### `mediaPlayer.media.search`

Searches within a collection. The `mediaId` scopes the search (e.g. search within a specific service or playlist). Returns a Collection object containing matching items.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Collection to search within. |
| `searchText` | string | Yes | Text to search for. |
| `inputId` | string | No | Input currently in use. When provided, the server uses the input's now-playing state to compute `canAddToQueue`. |
| `limit` | number | No | Maximum number of results to return. |
| `offset` | number | No | Zero-based start index for pagination. |
| `collectionActions` | number | No | See Collection actions. Default `0`. Applied only when `offset` is null or `0`. |

**Response data:** A single Collection object.

---

### `mediaPlayer.media.submitForm`

Submits a form in response to the user pressing a button. The client always POSTs using the pressed button's `id`. The body contains all form fields with user-supplied values. See Appendix A — Forms for the full interaction model.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `buttonId` | string | Yes | The `id` of the button that was pressed. |
| `fields` | array | Yes | Array of FormField objects — every field in the form, with `value` set to the user-entered value. Hidden fields must be included with their original server-provided value unchanged. |

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | No | Input currently in use. When provided, the server uses the input's now-playing state to compute `canAddToQueue` on the returned Collection. |

**Response data:** A Collection object on success. On cancellation the response contains `error.symbol == "FormCancel"` — the client should close the form and return to the previous browsing state. On a validation error the response contains an `error` with a user-displayable `message` — the client should display the message and keep the form open.

---

### `mediaPlayer.media.play`

Plays a specific media item on an input.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `inputId` | string | Yes | Target input. |
| `mediaId` | string | Yes | Media item to play (from featured, search, or queue). |
| `addToQueue` | string | Conditionally | How the item is added to the queue. See values below. Not required when selecting one of the items inserted via `collectionActions` (see Collection actions) — the server already knows the correct value for those. Required otherwise. |

**`addToQueue` values:**

| Value | Description |
|---|---|
| `playNow` | Replace queue with this item and play immediately. |
| `playShuffle` | Same as `playNow` but with shuffle enabled. |
| `playUnshuffle` | Same as `playNow` but with shuffle disabled. |
| `add` | Add item to the queue without changing playback. |
| `addplay` | Add item to the queue and start playing the first newly added item. |

**Request:**

```
{
  "id":   "31",
  "ns":   "mediaPlayer",
  "op":   "media.play",
  "args": { "inputId": "1", "mediaId": "12326726412843", "addToQueue":
```

```
  "playNow" }
  }
```

**Response data:** A single NowPlaying object.

---

### `mediaPlayer.media.refresh`

Triggers an asynchronous server-side refresh of a collection. The response is returned immediately; the actual refresh happens in the background. Only applicable when `canRefresh` is `true` on the collection or item.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Collection or item to refresh. |

**Response data:** `{ "success": true }`

---

### `mediaPlayer.media.delete`

Deletes a media item or collection. Only applicable when `canDelete` is `true`. After a successful delete the client should re-fetch the parent collection.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Item or collection to delete. |

**Response data:** `{ "success": true }`

---

### `mediaPlayer.media.rename`

Renames a media item or collection. Only applicable when `canRename` is `true`. After a successful rename the client should re-fetch the current collection to reflect the updated name.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Item or collection to rename. |
| `name` | string | Yes | New display name. |

**Response data:** `{ "success": true }`

---

### `mediaPlayer.media.setFeatured`

Bookmarks or un-bookmarks a media item or collection. Only applicable when `canFeature` is `true`.

**Request args:**

| Field | Type | Required | Description |
|---|---|---|---|
| `mediaId` | string | Yes | Item or collection to feature or un-feature. |
| `featured` | boolean | Yes | `true` to bookmark/feature; `false` to remove. |

**Response data:** `{ "success": true }`

---

### `mediaPlayer` Events

**`mediaPlayer.changed`** — Fired when the now-playing state of an input changes. `data` contains the complete NowPlaying object.

> **Progress bar guidance:** The `progress` field in this event is the authoritative re-sync point. Clients should not wait for server-sent progress updates — instead, maintain a local 1-second timer that increments `progress` automatically. The timer should run only when `status == 2` (playing) and `progressBar.isAvailable == true`. The displayed value must be clamped to `duration` and the timer stopped once `duration` is reached. For live or streaming sources, `duration` is `-1` and `progressBar.isAvailable` is `false` — do not run the timer. The `mediaPlayer.changed` event re-syncs progress whenever a track changes, playback starts or stops, or a seek/jump operation is performed.


```
{
  "type":  "event",
  "ns":    "mediaPlayer",
  "event": "changed",
  "data":  {
    "inputId":   "1",
    "mediaId":   "12326726412843",
    "metaData":  ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"],
    "status":    2,
    "duration":  318,
    "progress":  42,
    "featured":    { "isAvailable": true,  "value": "on" },
    "jump":        { "isAvailable": false },
    "next":        { "isAvailable": true,  "isEnabled": true },
    "pause":       { "isAvailable": true },
    "play":        { "isAvailable": false },
    "previous":    { "isAvailable": true,  "isEnabled": false },
    "progressBar": { "isAvailable": true },
    "queue":       { "isAvailable": true },
    "repeat":      { "isAvailable": true,  "value": "on" },
    "search":      { "isAvailable": true },
    "seek":        { "isAvailable": true },
    "shuffle":     { "isAvailable": true,  "value": "off" },
    "stop":        { "isAvailable": true },
```

```
      "thumbsDown":  { "isAvailable": true,  "value": false },
      "thumbsUp":    { "isAvailable": true,  "value": false },
      "hasQueueItems": false
    }
  }
```

---

`mediaPlayer.featured.changed` — Fired when the featured items for any input change. Clients should re-fetch via `mediaPlayer.featured.get` for the relevant input.

```
{
  "type":  "event",
  "ns":    "mediaPlayer",
  "event": "featured.changed"
}
```

---

## Subscriptions

Clients subscribe to topics using `core.subscribe`. A topic identifies a namespace or a specific resource within a namespace.

**Topic formats:**

| Topic | Description |
|---|---|
| `avSwitch` | All avSwitch zone events. |
| `avSwitch.zoneId` | Events for a specific zone only (future). |
| `mediaPlayer` | All mediaPlayer events. |
| `mediaPlayer.inputId` | Events for a specific input only (future). |
| `server` | All server events. |

> **Note:** Fine-grained topic filtering (e.g., `avSwitch.zoneId`) is reserved for future implementation. Currently, namespace-level subscriptions cover all events within that namespace.

Subscriptions persist for the lifetime of the WebSocket connection and are cleared automatically on disconnect.

---

## Data Types Reference

| Type | Description |
|---|---|
| string | JSON string. |
| number | JSON number (integer or float as indicated). |

| Type | Description |
|---|---|
| boolean | JSON `true` or `false`. |
| array | JSON array `[]`. |
| object | JSON object `{}`. |

**Volume range:** 0–100 (integer). Fractional values are not used.
**Timestamps:** ISO 8601 strings (e.g., `"2026-02-24T10:00:00Z"`).
**`id` format:** Any client-defined string. Recommended to be unique per session to simplify response correlation.

---

## Queue object

Returned by `mediaPlayer.queue.get`.

| Field | Type | Description |
|---|---|---|
| `startIndex` | number | Zero-based index of the first item in `items` relative to the full queue. |
| `totalAvailable` | number | Total number of items in the queue (for pagination). |
| `items` | array | Array of MediaItem objects. |

---

## Collection object

Returned by `media.getRoot`, `media.getCollection`, `media.search`, and `media.submitForm`.

| Field | Type | Description |
|---|---|---|
| `collectionId` | string | Identifier for this collection, usable with `media.getCollection`. |
| `title` | string | Display title for this collection. |
| `description` | string | Description of the collection. Omitted if unavailable. |
| `artworkUrl` | string | Artwork URL for this collection. Omitted if unavailable. |
| `canPlay` | boolean | Whether the entire collection can be played. |
| `canAddToQueue` | boolean | Whether the entire collection can be added to the queue. Requires `inputId` to be provided in the request; `false` if omitted. |
| `canSearch` | boolean | Whether this collection supports search. |
| `canRefresh` | boolean | Whether this collection can be refreshed. |
| `canDelete` | boolean | Whether this collection (or its items) can be deleted. |
| `canRename` | boolean | Whether this collection can be renamed. |
| `isFeatured` | boolean | `true` if this collection has been bookmarked/featured by the user. |
| `canFeature` | boolean | `true` if the user can bookmark/feature or un-feature this collection. |

| Field | Type | Description |
|---|---|---|
| `searchPromptText` | string | Hint text for a search input. Only present when `canSearch` is `true`. |
| `startIndex` | number | Zero-based index of the first item in `items` relative to the full collection. |
| `totalAvailable` | number | Total number of items available in the full collection (for pagination). |
| `items` | array | Array of MediaItem objects. |
| `form` | object | Form object to display for authentication or data entry. Omitted if not required. See Form object below. |
| `error` | object | Error detail if the collection could not be loaded. `{ "code": number, "message": string }`. Omitted on success. |

### MediaItem object

Represents a single item within a Collection.

| Field | Type | Description |
|---|---|---|
| `mediaId` | string | Identifier for this item, usable with `media.getCollection` (if browsable) or `media.play` (if playable). |
| `title` | string | Primary display title. Falls back to the first entry of `displayInfo` if not otherwise provided (e.g. for station-type items). |
| `details` | string | Secondary display text (e.g. artist, subtitle). Omitted if unavailable. |
| `artworkUrl` | string | Artwork URL. Omitted if unavailable. |
| `type` | number | Item type code. Determines whether the item is playable, browsable, or both. |
| `flags` | number | Capability flags bitmask. |
| `duration` | number | Track duration in seconds. `0` if not applicable. |
| `displayInfo` | array | Ordered array of display strings. Render in the order provided. |
| `isCollection` | boolean | `true` if this item is a browsable collection (can be opened with `mediaPlayer.media.getCollection`). |
| `isBrowseMore` | boolean | `true` if this item is a pagination sentinel — display as a "Browse more..." affordance rather than a regular item. To load the next page, call `mediaPlayer.media.getCollection` with the `mediaId` of this item and append the returned `items` to the existing collection item list. |

| Field | Type | Description |
|---|---|---|
| `isDisabled` | boolean | `true` if this item is unplayable or unavailable (deleted, permission denied, etc.). Display as greyed out. |
| `isFeatured` | boolean | `true` if this item has been bookmarked/featured by the user. |
| `canFeature` | boolean | `true` if the user can bookmark/feature or un-feature this item. |
| `canDelete` | boolean | `true` if this item can be deleted. |
| `canRefresh` | boolean | `true` if this item supports a refresh action. |
| `canDisplayInfo` | boolean | `true` if this item has secondary display text available (`details` is non-empty). |
| `canPlay` | boolean | `true` if this item itself can be played (selected). |
| `canAdd` | boolean | `true` if this item can be added to the queue. Takes into account the currently playing item's queue-type compatibility, the same way `canAddToQueue` does at the collection level — but unlike `canAddToQueue`, this ignores the collection's "disable bulk add" flag, since that only blocks adding an entire collection at once (e.g. not supported by Spotify), not adding this single item. |

---

### Form object

Returned inside a Collection when a service requires user input (e.g. authentication or search). Submission is driven by button IDs — there is no separate form-level ID.

| Field | Type | Description |
|---|---|---|
| `fields` | array | Array of FormField objects. May be empty for notice-style forms. |
| `buttons` | array | Array of FormButton objects. |

**FormField object:**

| Field | Type | Description |
|---|---|---|
| `key` | string | Field identifier, used as the key in `media.submitForm` pairs. |
| `title` | string | Display label. |
| `description` | string | Additional help text. Omitted if unavailable. |
| `type` | string | Input type (e.g. `"text"`, `"password"`, `"select"`). |
| `required` | boolean | Whether the field must be filled before submitting. |
| `value` | string | Pre-populated default value. Omitted if none. |
| `options` | array | Array of `{ "key": string, "value": string }` for `"select"` type fields. Omitted otherwise. |

**FormButton object:**

| Field | Type | Description |
|---|---|---|
| `key` | string | Button identifier (e.g. `"submit"`, `"cancel"`, `"signup"`). |
| `title` | string | Display label. |
| `id` | string | Endpoint identifier. Always present. The client POSTs to `media.submitForm` using this value as `buttonId` when the button is pressed. |
| `url` | string | External URL. Omitted if not applicable. When present, the client opens this URL in a new browser tab in addition to POSTing. |

---

## Appendix A — Forms

Forms allow music services to request user input. They are used for authentication (logging in to a streaming service) and for music service-specific operations such as adding a track to a playlist, or adding and removing tracks, albums, and playlists from a service's favorites. They are delivered as part of a Collection and require specific rendering and submission behaviour that differs from normal media browsing.

---

### When a Form is Present

When `collection.form` is non-null, the client must render the form UI instead of the media item list. The `items` array will be empty.

---

### Field Types

| Type | Rendering | Notes |
|---|---|---|
| `text` | Standard text input | |
| `password` | Text input with obscured characters | Must be cleared after every submission attempt |
| `select` | Single-choice dropdown | Build options from `field.options` — use `key` as the submitted value, `value` as the display label |
| `hidden` | Not rendered | Must be included in every submission with its original server-provided value unchanged |

Fields with a `value` property set by the server should be pre-populated in the UI.

---

### Validation

- Any button whose `key` is not `"cancel"` or `"signup"` must be **disabled** until all fields with `required: true` are non-empty.

- Button enabled/disabled state must update **reactively** as the user types — do not wait for an explicit validate action.
- `hidden` fields do not participate in validation; they are always submitted.
- Error messages from a previous submission attempt must be cleared when the user modifies any field.

---

### Submitting a Form

When the user presses any button, the client must:

1. **Always POST** via `media.submitForm` using the button's `id` as `buttonId`, regardless of which button was pressed (including cancel).
2. Send **all fields** in the form as the `fields` body — every field, in order, with `value` set to the user-entered value. Hidden fields are included with their original value.
3. If the button also has a `url`, **open that URL in a new browser tab** in addition to posting. This is used for OAuth and external signup flows.

---

### Handling the Response

| Condition | Client action |
|---|---|
| Response `error.symbol == "FormCancel"` | Close the form; return to the previous browsing state |
| Response has `error` (other symbol) | Display `error.message` inline; keep the form open for correction |
| Success (no error), button has `url` | The URL was already opened; close the form dialog |
| Success (no error), no `url`, returned Collection has `form` | Replace the current form with the new form and continue the interaction |
| Success (no error), no `url`, returned `collectionId` matches current view | Close the form; refresh the current collection view with the returned Collection |
| Success (no error), no `url`, returned `collectionId` differs from current view | Close the form; navigate to the returned Collection |

---

### Example: OAuth (Amazon Music Login)

Some services use OAuth rather than credential entry. The form has no input fields — only buttons. One button has both an `id` and a `url`.

**Button press:**

1. Client POSTs via `media.submitForm` using the button's `id`.
2. Client opens the button's `url` in a new browser tab, directing the user to the service's OAuth login page.

3. After the user authenticates externally, the service redirects back and the login completes. The client may receive a state-change event indicating the source is now authenticated.

---

## Appendix B — Change History

### 1.12.0 — 2026-07-12

- **Breaking:** `collectionActions` changed from a string enum (`None`/`PlayOnly`/`PlayShuffleOnly`/`All`) to a numeric bitmask (`1` = Play All, `2` = Shuffle All, `4` = Add To Queue, combinable). This lets a client request any combination directly — e.g. Shuffle All without Play All — instead of being limited to the four predefined combinations. See Collection actions.

### 1.11.0 — 2026-07-11

- MediaItem `canAdd` is now more accurate: it also reflects whether the currently playing item is queue-compatible, not just whether the item itself allows it.
- `addToQueue` is no longer required (and is ignored if sent) when calling `mediaPlayer.media.play` for one of the items inserted via `collectionActions` — the server already knows the correct action for those.
- MediaItem `title` is now always populated when available, falling back to `displayInfo` for items that don't have one (e.g. some station-type items).

### 1.10.0 — 2026-07-10

- Added `canPlay`/`canAdd` fields to the MediaItem object.
- `collectionActions: "All"` now also makes individually queueable items browsable, offering a "Play Now" / "Add To Queue" choice — useful for clients that only support single-tap selection.

### 1.9.0 — 2026-07-10

- `mediaPlayer.media.getCollection` now returns a "No items found" placeholder Collection instead of an empty or missing result when the underlying collection can't be found or has no items.
- Added the `collectionActions` request argument (`None` / `PlayOnly` / `PlayShuffleOnly` / `All`) to `media.getRoot`, `media.getCollection`, and `media.search`, automatically inserting "Play All" / "Shuffle All" / "Add All To Queue" items.

### 1.8.0 — 2026-07-08

- `server.get`'s `numberOfStreams` now reports the number of streams available (equal to the zone count) rather than the licensed count. The licensed count moved to a new `numberOfStreamsLicensed` field.

### 1.7.0 — 2026-05-10

- Collection and MediaItem model enhancements, and new media management commands (rename, delete, refresh, setFeatured, submitForm).

### 1.6.0 — 2026-05-06

- Added Sleep, Chimes, and TTS support to the WebSocket protocol.

**Earlier versions**

- Initial protocol draft and early revisions, including the original connection lifecycle, namespace/operation set, and the switch from server-pushed playback progress to client-computed progress (see State Reload and NowPlaying progress guidance).